CS6320 Assignment 1

GitHub Link

Group 22

Aankit Das
Net ID - axd220192

Kedar Rajendra Kadu
Net ID - kxk230024

1 Introduction

The report outlines the implementation of a language model using unigrams and bigrams. The dataset used in this assignment is a corpus of reviews of Chicago hotels. Initially, we preprocessed the data and cleaned it up for further operation. Firstly, we wrote a code to train unsmoothed unigrams and bigrams language models. Secondly, we implemented various smoothening techniques like Laplace Smoothening, Add-k smoothening, and Kneser-Ney Smoothening. Additionally, we also handled missing or unknown words from the dataset. Finally, we implemented the perplexity on the validation set for various models.

2 Implementation

2.1 Data Preprocesssing

We will be using the 'nltk' library for using basic language modeling tasks like tokenizing and using stopwords. For that, we downloaded and installed the 'nltk' package. Note that we have to download the 'punkt' and 'punkt_tab' modules for it to properly work.

Now, after downloading the required packages, we are ready to load and preprocess the data. For fast and efficient implementation, the data is loaded into a dataframe where each row contains one customer review. Next, a helper function is created which takes in a sentence as its argument and returns a filtered sentence. The function does the following preprocessing: removes punctuation, removes digits, tokenizes each word, performs stemming operation, removes stopwords, and joins each token to form a filtered sentence

Figure 1 shows the code snippet of the helper function that performs these tasks. Figure 2 shows the dataframe containing the actual reviews, and the preprocessed sentences side by side.

```python
#make a function to remove stop words and convert to lower case
def preprocessing(sentence):
    filtered_sent = sentence.translate(str.maketrans('', '', string.punctuation)) #remove punctuations
    filtered_sent = ''.join([char for char in filtered_sent if not char.isdigit()]) #remove digits
    word_tokens = word_tokenize(filtered_sent) #tokenize
    stem_tokens = [stemmer.stem(word) for word in word_tokens] #stemming
    filtered_tokens = [w for w in stem_tokens if not w.lower() in stop_words] #removing stopwords
    filtered_sent = ' '.join(filtered_tokens) #joining the tokens
    return filtered_sent
```

Figure 1: Preprocessing function.

```python
df['clean'] = df['reviews'].apply(lambda x: preprocessing(x))
df.tail()
```

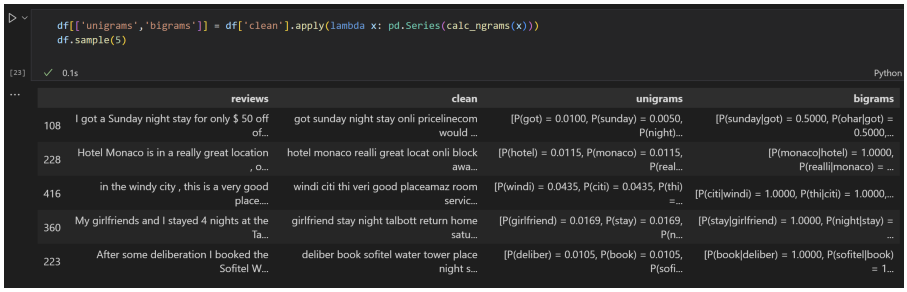
[215]

	reviews	clean
507	Swissotel continues to be a *yawn* As previous...	swissotel continu yawn previou poster state pu...
508	My husband & I stayed at the Fitzpatrick in ea...	husband stay fitzpatrick earli june birthdaygr...
509	I stayed at the Hilton Chicago last week and w...	stay hilton chicago last week wa veri disappoi...
510	Just back from 5 night stay at Omni and would ...	back night stay omni would thoroughli recommen...
511	We booked our hotel stay thru Yahoo and reques...	book hotel stay thru yahoo request room bed go...

Figure 2: Dataframe juxtaposing original and clean corpus.

2.2 Unigram and Bigram Probability Computation

In this section, we will see how to compute the unigram and bigram probabilities. The goal is to estimate the likelihood of words and word pairs based on their occurrences in the training dataset. For the unigram model, the probability of a word $P(w)$ is computed by dividing the number of times the word appears in the corpus by the total number of words. This gives us a measure of how likely any given word is to appear in isolation. For example, if the word 'book' appears 34 times in a corpus of 1000 words, its unigram probability would be $P(book) = 0.034$. For the bigram model, the probability of a word w_2 given that the previous word was w_1 is computed as $P(w_2|w_1)$, which is the frequency of the bigram (w_1, w_2) divided by the frequency of w_1. For instance, if the bigram 'book two' appears 10 times and 'book' appears 30 times in the corpus, the bigram probability would be $P(two|book) = 0.3$. Figure 3 shows the dataframe containing the unigrams and bigrams of the training corpus.



```
df[['unigrams','bigrams']] = df['clean'].apply(lambda x: pd.Series(calc_ngrams(x)))
df.sample(5)
```

[23] ✓ 0.1s Python

	reviews	clean	unigrams	bigrams
108	I got a Sunday night stay for only $ 50 off of...	got sunday night stay onli pricelinecom would ...	[P(got) = 0.0100, P(sunday) = 0.0050, P(night)...	[P(sunday|got) = 0.5000, P(ohar|got) = 0.5000,...
228	Hotel Monaco is in a really great location , o...	hotel monaco realli great locat onli block awa...	[P(hotel) = 0.0115, P(monaco) = 0.0115, P(real...	[P(monaco|hotel) = 1.0000, P(realli|monaco) = ...
416	in the windy city , this is a very good place....	windi citi thi veri good placeamaz room servic...	[P(windi) = 0.0435, P(citi) = 0.0435, P(thi) =...	[P(citi|windi) = 1.0000, P(thi|citi) = 1.0000,...
360	My girlfriends and I stayed 4 nights at the Ta...	girlfriend stay night talbott return home satu...	[P(girlfriend) = 0.0169, P(stay) = 0.0169, P(n...	[P(stay|girlfriend) = 1.0000, P(night|stay) = ...
223	After some deliberation I booked the Sofitel W...	deliber book sofitel water tower place night s...	[P(deliber) = 0.0105, P(book) = 0.0105, P(sofi...	[P(book|deliber) = 1.0000, P(sofitel|book) = 1...

Figure 3: Unigrams and Bigrams of test corpus.

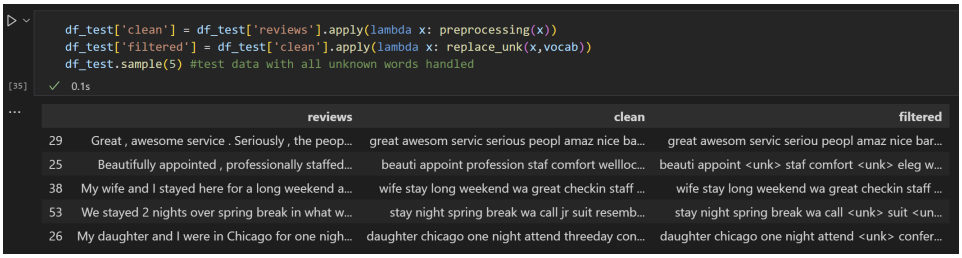
2.3 Smoothing

Smoothing is used in language models to handle situations where certain word combinations don't appear in the training data. Without smoothing, a model would assign a probability of zero to any unseen n-gram, which is problematic for calculating overall sentence probabilities (since multiplying by zero leads to zero). Some smoothing techniques used in this project are discussed as follows:

- **Laplace smoothing:** It addresses the issue of zero probabilities for unseen n-grams by adding a count of 1 to every possible n-gram, including those that were not observed in the training data. In this implementation, Laplace smoothing is applied to handle zero probabilities for unobserved bigrams. By adding a 1 to the observed bigram counts, the probability of any unobserved bigram becomes non-zero. This ensures that even for bigrams that do not appear in the training data, a small probability is assigned. This helps smooth the probabilities for both observed and unobserved bigrams, avoiding the problem of assigning zero probabilities to unseen data during testing.
- **Add-k smoothing:** It generalizes Laplace by allowing the addition of any constant k instead of just 1. The idea is to adjust how much probability mass is added to unseen n-grams by controlling k.
- **Kneser-Ney smoothing:** It improves upon previous methods by not only relying on observed n-gram counts but also by considering continuation probabilities—how likely a word appears in a new context.

2.4 Unknown Word Handling

To handle unknown words, that is, words that appear in the test dataset but are not seen in the training dataset, we first created a vocabulary from the training corpus. This vocabulary consists of all the words that appear frequently enough in the training data, defined by a threshold value (here we have selected 1). Any word in the training data that does not meet this threshold, is replaced by a special token $< unk >$. In practice, this means that for each word in the test corpus, we check if it exists in the vocabulary. If the word is missing, it is replaced by $< unk >$. Figure 4 shows the dataframe containing the filtered corpus after handling unknown words.



```
df_test['clean'] = df_test['reviews'].apply(lambda x: preprocessing(x))
df_test['filtered'] = df_test['clean'].apply(lambda x: replace_unk(x,vocab))
df_test.sample(5) #test data with all unknown words handled
```

[35] ✓ 0.1s

	reviews	clean	filtered
29	Great , awesome service . Seriously , the peop...	great awesom servic serious peopl amaz nice ba...	great awesom servic seriou peopl amaz nice bar...
25	Beautifully appointed , professionally staffed...	beauti appoint profession staf comfort wellloc...	beauti appoint <unk> staf comfort <unk> eleg w...
38	My wife and I stayed here for a long weekend a...	wife stay long weekend wa great checkin staff ...	wife stay long weekend wa great checkin staff ...
53	We stayed 2 nights over spring break in what w...	stay night spring break wa call jr suit resemb...	stay night spring break wa call <unk> suit <un...
26	My daughter and I were in Chicago for one nigh...	daughter chicago one night attend threeday con...	daughter chicago one night attend <unk> confer...

Figure 4: DataFrame illustrating the process of handling unknown words in the test corpus.

This allows the model to assign a probability to it, ensuring that the model can handle unseen words gracefully without assigning them zero probability. This process prevents computational issues such as $log(0)$ during perplexity calculations and provides the model with a fallback mechanism for rare or unseen words, improving the robustness of the language model.

2.5 Implementation of perplexity

Perplexity is an evaluation metric that provides a measure of how well a language model predicts a test corpus by quantifying the uncertainty of the model in predicting the next word in a sequence. Lower perplexity indicates better predictive capability of the model. We know that the perplexity for an n-gram model is calculated using the following formula:

$$PP = \exp\left(-\frac{1}{N}\sum_{i=1}^{N}\log P(w_i|w_{i-n+1},\ldots,w_{i-1})\right) \tag{1}$$

To compute perplexity, the model is first trained on the training dataset, where unigram and bigram probabilities are computed as discussed previously. Then, the perplexity calculation process for different models is discussed as follows:

- **Unigram Model:** For each word in the test dataset, we retrieve its probability from the precomputed unigram probabilities. If the word is unknown (i.e., not in the vocabulary), we replace it with the $< unk >$ token and use the probability of $< unk >$.
- **Bigram Model:** For each pair of words in the test dataset, we compute the bigram probability $P(w_2|w_1)$. If the bigram is not found in the training data, we fall back to the unigram probability of the second word to ensure the probability is non-zero. Again, unknown words are handled by using the $< unk >$ token.
- **Logarithmic Sum:** The logarithms of these probabilities are summed across all words or word pairs in the test set. The sum is then normalized by dividing it by the total number of words in the test set.
- **Perplexity:** The perplexity is computed by taking the exponent of the negative normalized log probability sum. This value reflects how well the model predicts the test corpus, with a lower perplexity indicating better predictive performance.

Figure 5 shows the code snippet for perplexity calculations for both unigrams and bigrams.

In the following section, we will see how this metric is essential for evaluating language models. The unigram model typically results in higher perplexity values, indicating that it lacks the contextual understanding offered by the bigram model, which incorporates word pairs.

3 Evaluation, Analysis, and Findings

In this subsection, we evaluate the performance of the unigram and bigram models based on their predictive capabilities and how well they handle word sequences. The evaluation focused on the relevance of the next-word predictions generated by each model. Figure 6 shows the output of both the models given a test input sentence which is incomplete.

We can clearly see that the bigram model leverages the preceding word to make a more informed prediction and is coherent with the input sentence. This context-dependent approach enhances

```
if model == "unigram":
    for word in tokens:
        prob = unigram_probs.get(word, unigram_probs.get("<unk>", 1e-10))  #
        log_prob_sum += math.log(prob)

elif model == "bigram":
    for i in range(1, len(tokens)):
        w1, w2 = tokens[i - 1], tokens[i]

        #check for bigram probability first
        if w1 in bigram_probs and w2 in bigram_probs[w1]:
            prob = bigram_probs[w1][w2]
        else:
            #fallback to unigram probability or <unk>
            prob = unigram_probs.get(w2, unigram_probs.get("<unk>", 1e-10))
        log_prob_sum += math.log(prob)

perplexity = math.exp(-log_prob_sum / total_words)
return perplexity
```

Figure 5: Code snippet showing perplexity calculations for the language models.

```
test_snippet = "The hotel has a great "
#predict the next word using the unigram model
next_word_unigram = predict_next_word_unigram(unigrams_dict)

#predict the next word using the bigram model
last_word = test_snippet.split()[-1]
next_word_bigram = predict_next_word_bigram(last_word, bigrams_dict, unigrams_dict)

#output the results
print(f"Unigram model prediction: {next_word_unigram}")
print(f"Bigram model prediction: {next_word_bigram}")
[86]  ✓ 0.0s

...  Unigram model prediction: enjoy
     Bigram model prediction: view
```

Figure 6: Output of unigram and bigram model.

the model's ability to generate coherent and contextually appropriate predictions, thereby improving the overall quality of text generation. On the other hand, the unigram model predicted the most frequently occurring word regardless of the actual context. This lack of contextual awareness demonstrates a significant limitation of the unigram approach, which fails to capture the relationships between words.

These findings are further substantiated by the evaluation metric known as perplexity. Lower perplexity values indicate that the model has a better understanding of the word sequences in the test data. The unigram model typically exhibited higher perplexity scores, reflecting its simplistic nature and lack of context consideration. In contrast, the bigram model demonstrated lower perplexity values, underscoring its ability to capture relationships between consecutive words and better predict the test data.

To illustrate this, we computed the perplexity on a held-out test dataset using both models. The results (see Figure 7) revealed that the bigram model outperformed the unigram model, reinforcing the notion that incorporating contextual information significantly enhances predictive performance. We computed the perplexity after applying all the smoothing techniques. Figure 8 shows the table containing the perplexity values of all the models. We can see that the Unigram model has a relatively high perplexity of 15.97, meaning it struggles to predict the next word accurately. The Bigram model significantly improves on this with a perplexity of 8.00. Different smoothing techniques are also tested: Laplace Smoothing marginally increases the perplexity to 8.89, Add-k Smoothing performs the best with the lowest perplexity of 7.92, indicating it handles unseen n-grams effectively. We observe that Kneser-Ney Smoothing has a perplexity of 27.20, which is surprisingly high compared to its expected performance in handling sparse data. It could be because the model needs more data or the dataset does not have enough repeating patterns.

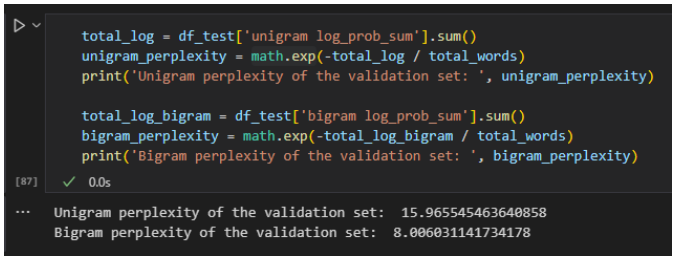


Figure 7: Perplexity values of both unsmoothed unigrams and bigrams.

	Model	Perplexity
0	Unigram	15.97
1	Bigram	8.00
2	Laplace Smoothing	8.89
3	Add-k Smoothing	7.92
4	Kneser-Ney Smoothing	27.20

Figure 8: Perplexity values of all the models.

4 Details of Programming Library

This assignment is entirely done using Python and therefore several Python libraries were utilized to facilitate the tasks. Pandas was the primary library used for DataFrame manipulation and for efficient handling and transformation of structured data. For managing and combining dictionaries, working with collections of data, and performing mathematical operations, Python's built-in libraries such as 'collections', and 'math' were used. Additionally, we used the 'NLTK' library which is one of the most popular and comprehensive tools for NLP. We used it for tokenization, stemming, and stopword removal.

5 Workflow

We have divided the workflow into three broad categories. The first part of the assignment, that is, data preprocessing, and calculating unigrams and bigrams, is done by Aankit Das. The second part involving the task of unknown word handling and exploring various smoothening techniques, is done by Kedar Kadu. The final part, perplexity calculation is done by both Aankit Das and Kedar Kadu so that we both have a comprehensive look at the evaluation of the models.

6 Feedback for the Project

The project was a good balance of difficulty and understanding. It required a solid grasp of concepts like n-gram models, probability computation, perplexity, and dataset handling. Loading the data into an appropriate data structure and performing proper preprocessing significantly highlighted the importance of preparing raw data for further analysis. A significant part of the project involved converting theoretical formulas related to n-gram probability calculations into functional code, which proved to be a time-consuming process. This step involved multiple iterations and debugging to ensure the code was readable and robust. It ultimately helped bridge the gap between theory and application.

By implementing both the unigram and bigram models, we gained a clearer perspective on the importance of context in language prediction. The task involved several components, from preprocessing data to evaluating model performance, and debugging, which took a considerable amount of time—around 25 to 30 hours in total. The only suggestion would be to provide more structured guidance on smoothing techniques, especially the Kneser-Ney technique as these were particularly challenging. Overall, the project was valuable for reinforcing theoretical knowledge through practical application.